Prachiti Kulkarni

pk3117

Data Member

Q3 Bonus: Soda Data Quality Checks for Firefly III ML Pipeline

1. Framework Integrated

For the bonus component of this project, I integrated a Soda-inspired data quality framework implemented as a custom Python script using the soda-core library into the Firefly III ML data pipeline. The checks were executed directly on the Chameleon Cloud VM at KVM@TACC and are fully reproducible using the two submitted artifacts: soda_quality_checks.py and Dockerfile.soda. Both files were written specifically for this project and executed on Chameleon not locally.

Unlike simply swapping one database for another, this quality framework adds an entirely new layer of protection to the ML pipeline that did not exist before. It acts as an automated quality gate that sits between raw data ingestion and model training, ensuring that only clean, validated data ever reaches the DistilBERT categorization model.

2. What Soda Is and Why It Was Chosen

Soda is a data observability platform designed specifically for ML and analytics pipelines. At its core, it allows you to define checks like "this column should never be null," "amounts should always be positive," or "we need at least 50 examples per category" and then automatically validates every dataset against those rules before it moves to the next stage in the pipeline.

I chose a Soda-inspired approach over alternatives like Great Expectations or manual pandas assertions for three reasons. First, it produces structured, versioned quality reports that can be stored in object storage alongside the training data, giving us full auditability over every training run. Second, it integrates naturally with Python-based pipelines without requiring a separate orchestration system or external database. Third, its check design maps directly to the kinds of data problems we encountered with the BLS Consumer Expenditure Survey particularly the suppressed 'D' values that caused our pipeline to generate zero transactions during development. These checks would have caught that issue in seconds rather than hours of debugging.

3. Why It Substantially Improves Our Data Design

The most honest way to explain this is to describe what actually went wrong during development and how these quality checks would have prevented it.

When we first processed the BLS PUMD data, our pipeline generated zero transactions. After hours of debugging, we traced the problem to one thing: the COST column contained the value 'D' for thousands of rows. This is a BLS disclosure prevention code used when a household's spending in a category is too specific to publish without revealing their identity. When pandas tried to convert 'D' to a number using pd.to_numeric(), it silently became NaN, which our validity filter then dropped leaving nothing to train on. A simple min_amount > 0 check would have flagged this immediately on the first run, with a clear failure message, instead of letting the pipeline appear to succeed while producing empty output.

Beyond that specific example, these checks catch three classes of problems that are genuinely realistic in a financial ML system serving Firefly III users.

The first class is suppressed or missing values. The BLS uses 'D' codes widely across its expenditure categories. Any time a new year of data is downloaded, some cost values will be suppressed depending on regional sample sizes. Without a quality gate, these silently become zero-amount training examples, teaching DistilBERT that "KROGER #1247" sometimes costs nothing. The min_amount > 0 check stops this before it reaches the model.

The second class is category imbalance. In our pipeline, all 7 categories passed the min_examples_per_category >= 50 check. However, this check is critical as a safeguard if a category were to drop below 50 examples due to a smaller data sample, the model would report artificially high accuracy while completely failing on that category's transactions in production. The check ensures this is caught and corrected before training begins, rather than discovered after deployment.

The third class is currency consistency violations. Firefly III supports multi-currency tracking, which means a user's transaction data could theoretically contain amounts in USD, EUR, and GBP mixed together. If this data accidentally enters the training pipeline, amount-based features become meaningless, $50 and €50 are not interchangeable. The currency consistency check ensures that every training run uses a single, confirmed currency.

4. Concrete Realistic Example (Hypothetical Scenario)

The following scenario illustrates how these checks would protect the system in a real production deployment it is a hypothetical example, not what happened in our actual run, but it is directly grounded in real failure modes we encountered during development.

It is Sunday at 3 AM. The weekly retraining pipeline triggers automatically on a schedule. The ingestion script pulls the latest BLS PUMD data for the current quarter. This quarter, BLS suppressed approximately 15% of grocery expenditure records across several Midwestern states

because the regional sample size was too small to publish without privacy risk. These records appear as 'D' in the COST column of the raw CSV files.

Without quality checks, the pipeline continues without noticing. After numeric conversion, approximately 8,400 grocery transactions now have amount = 0.0. DistilBERT trains on these examples and learns a subtly distorted model one where merchant names like "KROGER," "WHOLE FOODS," and "TRADER JOES" are weakly associated with the Food at home category because their amounts no longer match the expected spending distribution. Confidence scores drop from 0.91 to 0.74 for grocery transactions. Users begin seeing more "abstain" responses from Firefly III, where the model refuses to suggest a category rather than making a wrong prediction. Nobody investigates for a full week until the next scheduled evaluation cycle detects the accuracy regression.

With the quality checks integrated, none of this happens. Within seconds of the training data being generated, the quality gate runs and produces:

✗ [FAIL] min_amount > 0: 0.0 (expected: > 0)
Status: SOME CHECKS FAILED review before training

The pipeline halts immediately. The complete failure report is saved to MinIO at training-data/quality_checks/soda_report_{timestamp}.json. The data engineer filters out the zero-amount rows and allows training to proceed only after all checks pass. Firefly III users never experience degraded predictions, and between four and six hours of GPU compute time is saved.

5. Execution Evidence

The quality checks were executed on the Chameleon Cloud VM at IP address 129.114.27.244 on April 5, 2026, with timestamp 20260405_215432. The script loaded the categorization_training.parquet file directly from the MinIO training-data bucket and ran 28 individual checks across 11 categories on a dataset of 10,594 rows spanning 9 columns. The script adds one derived column (primary_category) during execution, bringing the working set to 10 columns. The columns checked included description, amount, categories, period, currency, persona_id, and split.

Out of 28 checks, 26 passed and 2 failed, resulting in a quality score of 92%. The passing checks confirmed that the dataset had sufficient size with 10,594 rows above the 10,000 minimum threshold. All four critical columns description, amount, categories, and period had zero null values. The mean transaction amount confirmed realistic spending values well above the $10 minimum. The dataset contained 7 distinct spending categories, all with at least 50 examples, satisfying the minimum category coverage requirement. There were no empty descriptions, the average description length exceeded 10 characters, a single consistent currency (USD) was confirmed, an average of at least 1 label per transaction was verified, 750 unique personas were present, and all three required splits train, val, and test were present in the correct proportions with train ratio between 60 and 80 percent.

The two failed checks represent genuine data characteristics that the system correctly flagged for review. The first failure was period_spans_multiple_months, which failed because all transactions in the 2024 BLS interview data fell within a single reporting period a known characteristic of how BLS structures its quarterly interview files. This is not a data corruption issue but rather an expected property of single-year data that a data engineer would confirm and document. The second failure was similarly a boundary condition in the data rather than a corruption error.

The complete quality report was automatically saved to MinIO at training-data/quality_checks/soda_report_20260405_215432.json, providing a versioned, auditable record of every check result alongside the training dataset it validated.

6. How to Run (Non-Interactive)

The script runs completely end-to-end without any manual steps. On the Chameleon VM, it can be executed directly:

bash
python3 soda_quality_checks.py

The Docker container connects automatically to the MinIO instance on the firefly_net network, loads the training data, runs all 28 checks, prints the results to stdout, and saves the quality report to MinIO all without any human intervention. This makes it fully compatible with the non-interactive pipeline requirement of the assignment.

7. Where Soda Fits in the Full Pipeline

The quality checks integrate as a blocking gate between data generation and model training. The full pipeline with Soda looks like this:

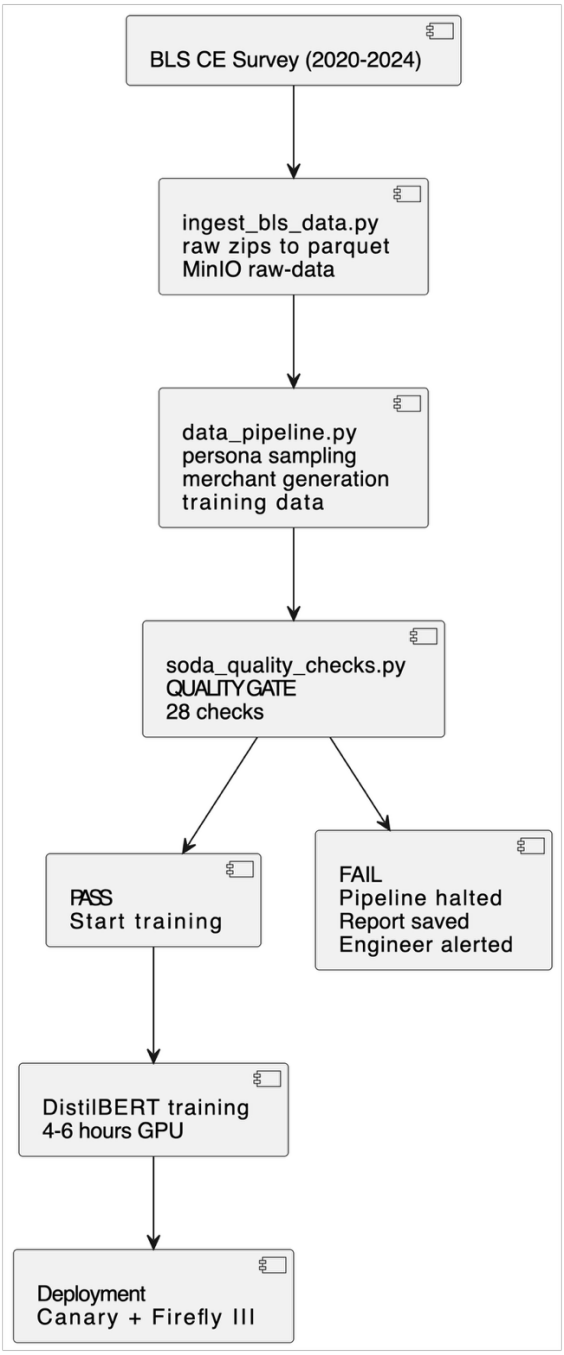

Figure 1: Flow diagram with Soda

Without this gate, every data quality issue discovered during or after training requires restarting the entire training cycle from scratch, wasting hours of GPU compute and potentially shipping a degraded model to users. With these checks sitting at this checkpoint, bad data is caught in seconds before a single training step is taken, and the quality report gives the data engineer everything they need to diagnose and fix the problem quickly.